BATTAGLIA NAVALE

Algoritmi Distribuiti 2025/2026

Alex Billa

Contents

1 Software Requirements Specification (SRS) 2
1.1 Scopo e contesto . . . 2
1.2 Requisiti funzionali . . . 2
1.3 Requisiti non funzionali . . . 3
1.4 Attori del sistema . . . 3

2 Architettura del sistema 4
2.1 Panoramica architetturale . . . 4
2.2 Componenti principali . . . 5
2.3 Principi distribuiti implementati . . . 5
2.3.1 Callback Architecture) . . . 5
2.3.2 Stato distribuito (Board ownership) . . . 5
2.3.3 Negoziazione della configurazione . . . 5
2.3.4 Pattern Observer per il disaccoppiamento UI/rete . . . 5
2.4 Struttura dei file . . . 6

3 Protocolli di comunicazione 7
3.1 Connessione iniziale e negoziazione configurazione . . . 7
3.2 Piazzamento navi e avvio partita . . . 8
3.3 Partita: ciclo di gioco . . . 8
3.4 Rivincita . . . 9
3.5 Riepilogo di tutte le chiamate remote . . . 10

1. Software Requirements Specification (SRS)

1.1 Scopo e contesto

Il progetto consiste nell'applicazione di una versione distribuita del gioco **Battaglia Navale**, in cui due client si connettono a un server e giocano tra loro in tempo reale. L'obiettivo tecnico principale è dimostrare i principi fondamentali degli **algoritmi distribuiti**: comunicazione bidirezionale tra nodi, gestione dello stato distribuito e coordinazione senza conoscenza globale del sistema.

Il sistema è sviluppato in Python con il middleware `Pyro5` (*Python Remote Objects*), che permette di invocare metodi su oggetti remoti in modo trasparente attraverso una connessione TCP/IP. Ogni processo (server e client) espone un oggetto remoto registrato su un *Daemon* `Pyro5` e identificato da un URI univoco.

Ogni client è un nodo attivo della rete, espone un proprio endpoint Pyro5 raggiungibile dal server, e detiene in maniera esclusiva lo stato della propria griglia di gioco. Il server non ha conoscenza sull'intero sistema, ma si occupa di accettare due client per avviare la partita, ricevere le mosse dei client e passarle al rispettivo avversario, gestire i turni e la eventuale rivincita.

1.2 Requisiti funzionali

Table 1: Requisiti funzionali del sistema

ID	Requisito
RF-01	Due client si connettono al server tramite `Pyro5` NameServer
RF-02	I giocatori come prima cosa *negoziano* la configurazione delle navi tramite un ciclo proposta / accetta o contro-proposta
RF-03	Il server coordina i turni e instrada le comunicazioni tra i due nodi
RF-04	I client, a turno, iniziano a sparare sulla griglia avversaria
RF-05	Ogni client riceve il colpo dell'avversario e calcola il risultato da inviare al server
RF-06	Ogni client mostra la propria griglia e la griglia di tracciamento dei colpi sparati all'avversario
RF-07	Viene mostrato lo stato di ogni nave (intatta / colpita / affondata) durante la partita
RF-08	A fine partita i giocatori possono votare per una rivincita; se entrambi accettano la partita ricomincia senza riconnettersi
RF-09	Il client supporta due modalità di gioco: terminale e GUI Tkinter, selezionabili da riga di comando

1.3 Requisiti non funzionali

Table 2: Requisiti non funzionali del sistema

ID	Requisito	Categoria
RNF-01	I client possono scegliere se utilizzare la versione da terminale o quella con GUI	Usabilità
RNF-02	Ogni client si occuperà di eseguire il calcolo dell'esito dei colpi sparati e rilevare se la partita è finita, senza centralizzare tutto sul server	Performance
RNF-03	Nessun polling: le notifiche avvengono esclusivamente via callback push (`Pyro5`)	Performance
RNF-03	Il server non conosce mai la posizione delle navi (privacy dello stato locale)	Sicurezza
RNF-04	I proxy `Pyro5` sono creati *on-demand* nel thread corrente per rispettare il vincolo di thread ownership	Affidabilità
RNF-05	GUI e daemon `Pyro5` girano su thread separati per evitare blocchi del main thread	Usabilità
RNF-06	La logica di rete è separata dalla logica di interfaccia tramite il pattern **Observer** (`ClientCallback`)	Manutenibilità
RNF-07	Il sistema supporta il riavvio di una partita senza dover riconnettersi al server	Usabilità

1.4 Attori del sistema

Table 3: Attori del sistema

Attore	Tipo	Responsabilità
Giocatore A / B	Utente umano	Interagisce con il sistema via terminale o GUI
Client A / B	Nodo attivo `Pyro5`	Espone oggetto remoto; gestisce Board in locale; calcola esito colpi ricevuti
Server	Coordinatore `Pyro5`	Gestisce turni, log, negoziazione config, routing, voto rivincita
NameServer	Registro `Pyro5`	Risolve nomi simbolici (*battleship.server*) in URI di rete

2. Architettura del sistema

2.1 Panoramica architetturale

Il sistema adotta un'architettura **ibrida client-server con callback bidirezionale**. Il server svolge il ruolo di coordinatore centralizzato per la gestione dei turni e del log di partita, ma *non è l'unico nodo attivo*: ogni client espone a sua volta un oggetto `Pyro5` che il server può chiamare direttamente. Questo rende i client nodi a pieno titolo della rete distribuita.

Il **NameServer** `Pyro5` funge da servizio di directory: il server ad esempio registra il proprio URI con il nome simbolico `battleship.server`; i client lo interrogano per ottenere l'indirizzo del server senza hardcoding dell'IP o della porta.

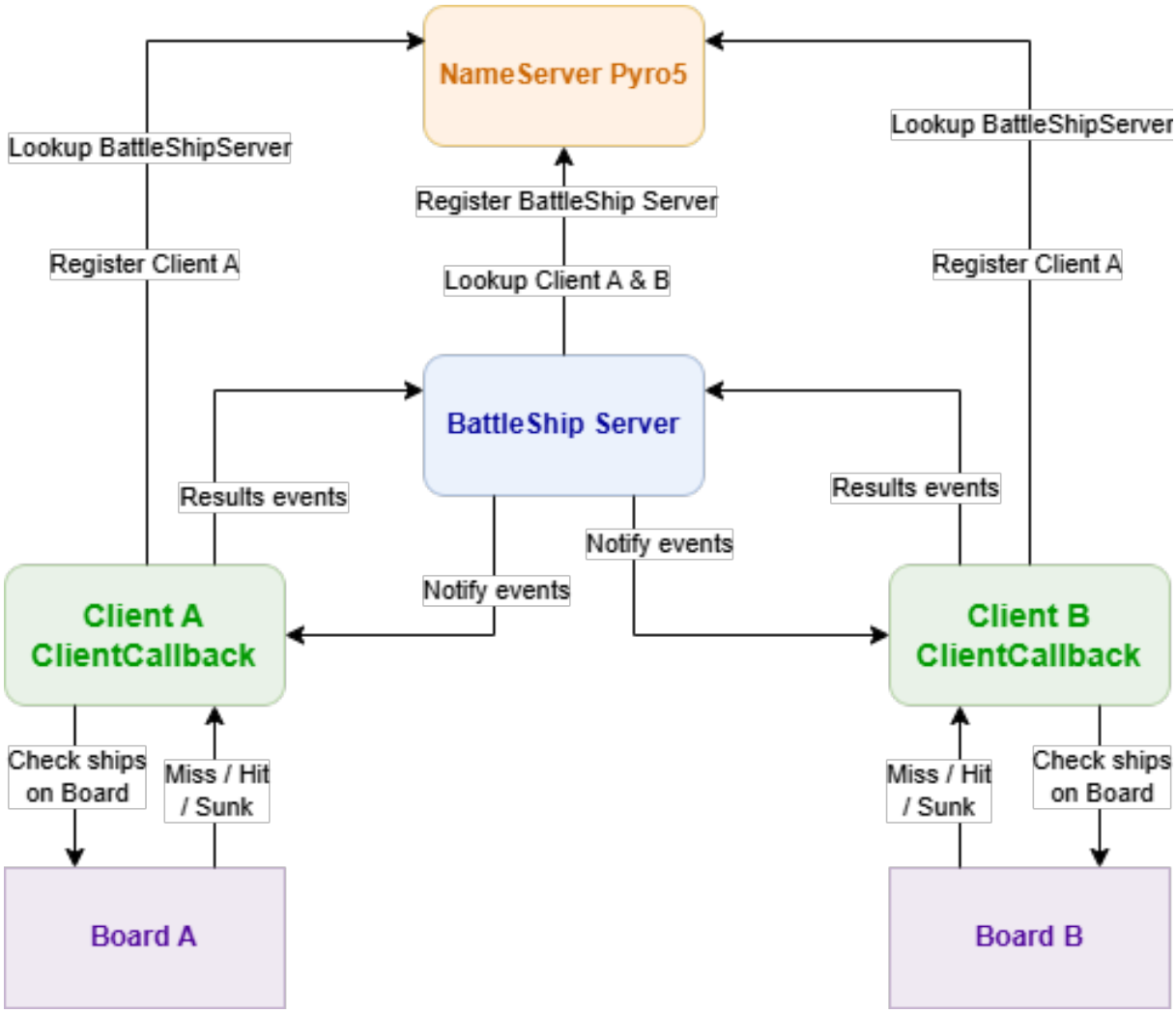

Figure 1: Architettura del sistema distribuito.

2.2 Componenti principali

Table 4: Componenti del sistema e relative responsabilità

Componente	Processo	Responsabilità
`BattleshipServer`	Server	Gestione turni, routing chiamate, log partita, negoziazione config, voto rivincita
`ClientCallback`	Client	Oggetto `Pyro5` esposto: `receive_shot`, `notify_*`, `ping`
`Board`	Client	Griglia 10×10, calcolo esito colpi, tracking grid
`Ship`	Client	Stato singola nave: celle occupate, colpi ricevuti, affondamento
`client_terminal`	Client	Hook Observer → `threading.Event`; loop input da terminale
`BattleshipGUI`	Client	Hook Observer → `root.after()`; interfaccia Tkinter
`presets`	Entrambi	Configurazioni navi predefinite

2.3 Principi distribuiti implementati

2.3.1 Callback Architecture)

Ogni client avvia un proprio *Daemon* `Pyro5` su un thread separato e ne passa l'URI al server in fase di registrazione. Il server costruisce un proxy verso ciascun client **on-demand** — creando un nuovo proxy per thread per rispettare il vincolo di ownership `Pyro5` — e lo chiama direttamente per inviare delle notifiche: assegnazione del turno, esito del colpo, fine partita. In questo modo è stato evitato il continuo polling che avrebbe generato un carico sul sistema più alto, andando a implementare invece una gestione tramite eventi.

2.3.2 Stato distribuito (Board ownership)

Il server non conosce la posizione delle navi. Quando deve valutare un colpo, chiama `receive_shot(coord)` sul nodo difensore: è il client stesso a calcolare l'esito sul proprio stato locale e a restituire solo la stringa risultato (`miss / hit / sunk:Nome / sunk_all:Nome`). Il server agisce da arbitro senza detenere lo stato di gioco completo.

2.3.3 Negoziazione della configurazione

La scelta della configurazione delle navi avviene tramite un ciclo di negoziazione tra i due client, mediato dal server, che segue questo flusso:

1. Un client propone una configurazione.
2. Il server la inoltra all'avversario.
3. L'avversario accetta oppure fa una contro-proposta.
4. Il ciclo si ripete finché uno dei due non accetta.
5. Il server finalizza e distribuisce la configurazione concordata (`notify_ship_config`) a entrambi i nodi.

2.3.4 Pattern Observer per il disaccoppiamento UI/rete

`ClientCallback` espone le hook `on_*` che vengono impostate dal livello UI *prima* della connessione. Le callback `Pyro5` arrivano nel thread del daemon e vengono inoltrate alle hook:

- **Terminale**: le hook settano `threading.Event`, che sblocca il main thread in attesa di input.

- **GUI**: le hook chiamano `root.after(0, fn)`, che schedula l'aggiornamento widget sul main thread Tkinter in modo thread-safe.

2.4 Struttura dei file

Table 5: File del progetto

File	Descrizione
`server.py`	`BattleshipServer`: coordinatore `Pyro5`
`client.py`	Entry point (`--gui` / `--term`)
`client_callback.py`	`ClientCallback`: oggetto `Pyro5` + hook Observer
`client_terminal.py`	Modalità terminale (`threading.Event`)
`client_gui.py`	Modalità GUI Tkinter (`root.after`)
`board.py`	`Board`: griglia, `receive_shot`, tracking
`ship.py`	`Ship`: stato nave, colpi, affondamento
`presets.py`	Configurazioni preset delle navi

3. Protocolli di comunicazione

Tutti i messaggi sono **invocazioni di metodi remoti** su connessione TCP/IP. Nei diagrammi di sequenza che seguono:

- **Freccia Blu** (⟶) — chiamata da Client verso Server.
- **Freccia Verde** (⟶) — chiamata callback da Server verso Client.
- **Freccia viola** (⟶) — chiamate da e verso Board.

3.1 Connessione iniziale e negoziazione configurazione

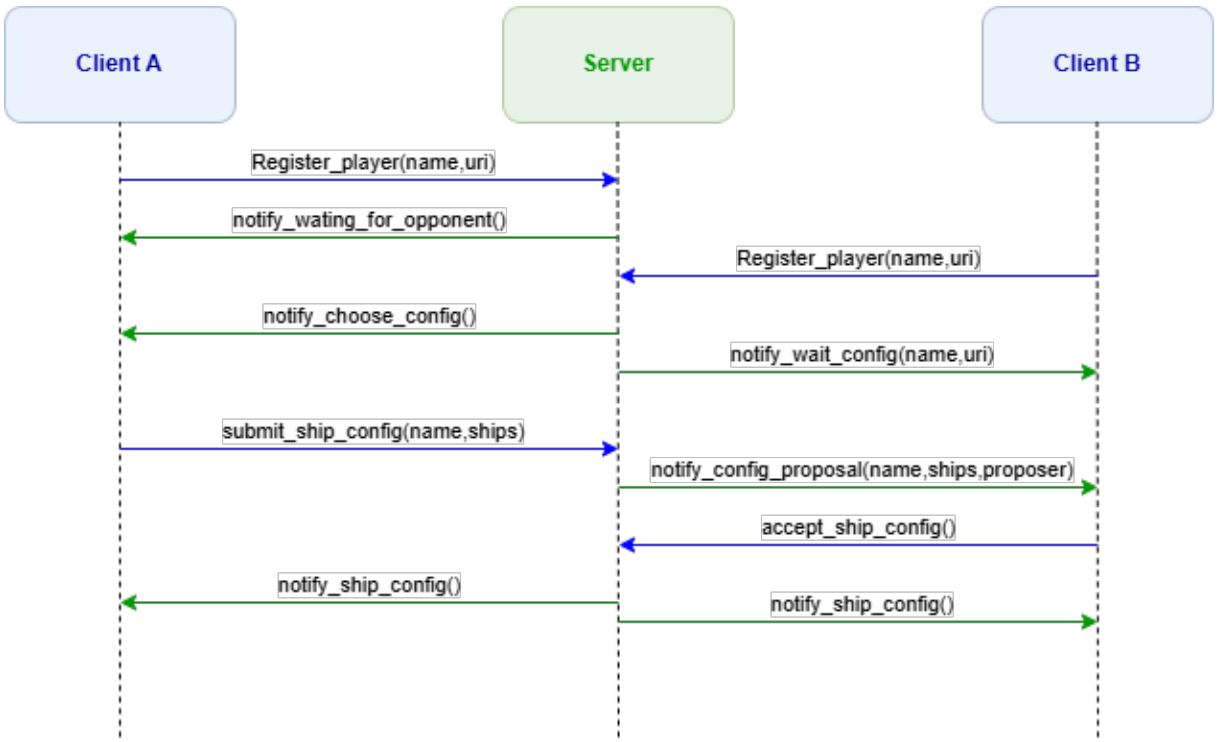

Figure 2: Diagramma di sequenza UML — Connessione e negoziazione della configurazione delle navi.

Appena entrambi i client si sono connessi, il server avvia il ciclo di negoziazione: Client A propone una configurazione tramite `submit_ship_config`; il server la inoltrata a Client B come proposta. Client B può accettare (`accept_ship_config`) o fare una contro-proposta. Il ciclo termina quando uno dei due accetta: il server chiama `notify_ship_config` su entrambi e i client passano al piazzamento.

3.2 Piazzamento navi e avvio partita

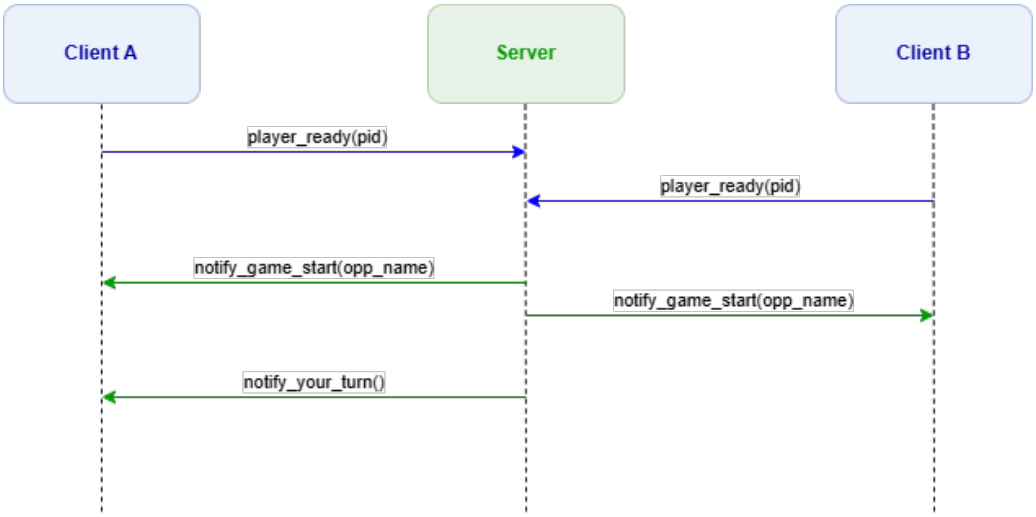

Figure 3: Diagramma di sequenza UML — Piazzamento navi e avvio partita.

Una volta che il giocatore finisce di piazzare le proprie navi, il client chiama `player_ready` per comunicare al server che l'utente è pronto a giocare. Quando entrambi i client chiamano player_ready, il server avvia la partita notificando entrambi con `notify_game_start` e assegnando il primo turno con `notify_your_turn` al primo giocatore che si è connesso al server.

3.3 Partita: ciclo di gioco

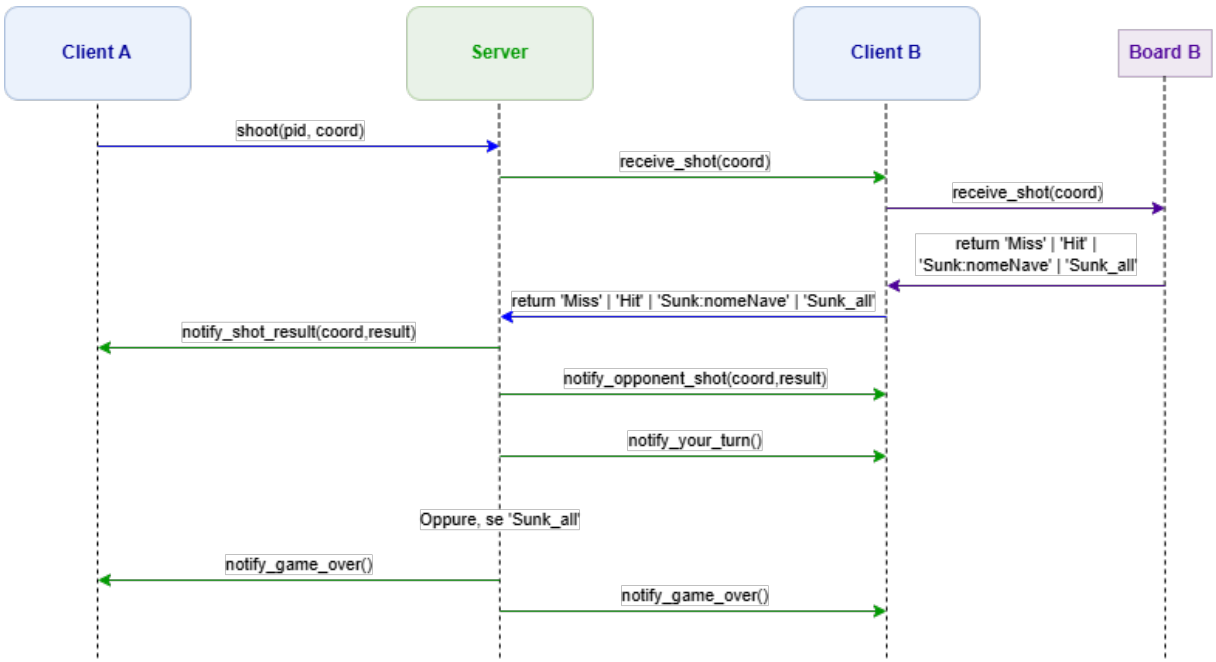

Figure 4: Diagramma di sequenza UML — Una mossa di gioco completa.

Ogni volta che un client comunica al server il proprio colpo da sparare, il server delega il calcolo dell'esito al client avversario. Il difensore è l'unico che conosce la posizione delle proprie navi (il rispettivo oggetto della classe Board contiene queste informazioni); al server ritorna solo la stringa risultato, descritta nella Tabella 6.

Table 6: Valori di ritorno di `receive_shot`

Valore	Significato
`miss`	Il colpo ha mancato tutte le navi
`hit`	Il colpo ha colpito una nave (non ancora affondata)
`sunk:NomeNave`	Il colpo ha affondato la nave *NomeNave*
`sunk_all:NomeNave`	Tutte le navi sono affondate: la partita termina
`already_shot`	La cella era già stata colpita in precedenza
`invalid`	Coordinata non valida (errore di formato)

3.4 Rivincita

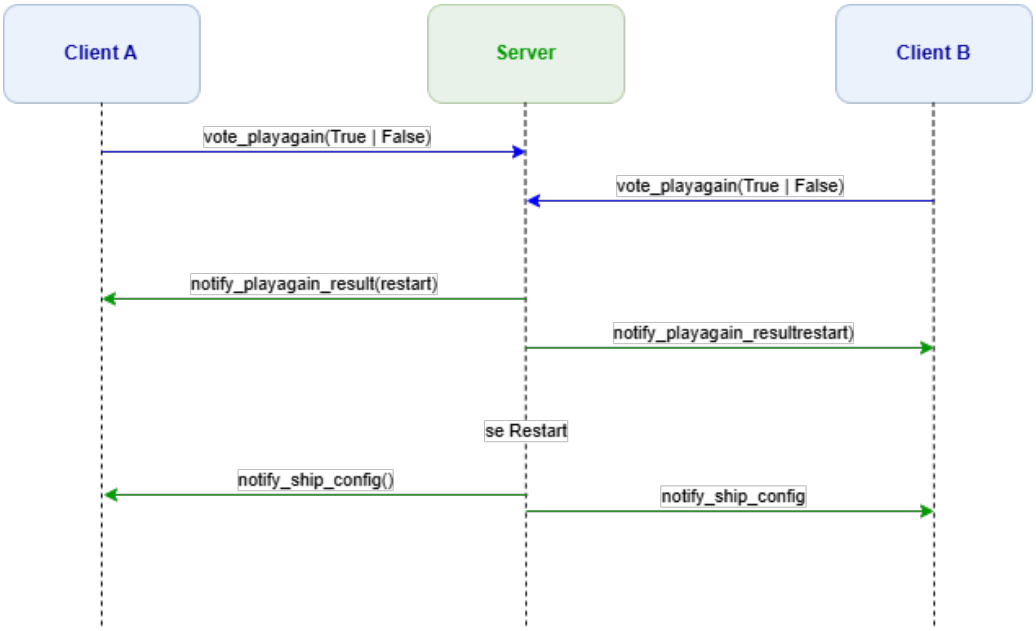

Figure 5: Diagramma di sequenza UML — Voto rivincita.

Il server raccoglie i voti di entrambi i client. Se entrambi votano `True`, chiama internamente `_reset_for_new_game()`: azzera turni, log, voti e stato `ready`, poi ritrasmette la configurazione già concordata tramite `notify_ship_config`. I client tornano automaticamente alla fase di piazzamento senza doversi riconnettere.

3.5 Riepilogo di tutte le chiamate remote

Table 7: Riepilogo completo delle chiamate remote `Pyro5`

Chiamante	Destinatario	Metodo (firma)	Fase
Client	Server	`register_player(name, uri)`	Connessione
Client	Server	`submit_ship_config(pid, name, ships)`	Negoziazione
Client	Server	`accept_ship_config(pid)`	Negoziazione
Client	Server	`player_ready(pid)`	Piazzamento
Client	Server	`shoot(pid, coord)`	Partita
Client	Server	`vote_play_again(pid, bool)`	Rivincita
Server	Client	`notify_waiting_for_opponent()`	Connessione
Server	Client	`notify_choose_config()`	Negoziazione
Server	Client	`notify_wait_config()`	Negoziazione
Server	Client	`notify_config_proposal(name, ships, proposer)`	Negoziazione
Server	Client	`notify_ship_config(name, ships)`	Negoziazione
Server	Client	`notify_game_start(opp_name)`	Partita
Server	Client	`receive_shot(coord) → result`	Partita
Server	Client	`notify_your_turn()`	Partita
Server	Client	`notify_shot_result(coord, result)`	Partita
Server	Client	`notify_opponent_shot(coord, result)`	Partita
Server	Client	`notify_game_over(won)`	Partita
Server	Client	`notify_play_again_result(restart)`	Rivincita